

Assignment 1

1

More powerful instructions lead to higher performance since the total number of instructions executed is smaller for a given task with more powerful instructions.

✓ **Answer** ✓

False, not always true, larger instructions normally take more time to execute meaning you have to run at a slower clock speed.

2

MIPS has two basic data transfer instructions, including lw and sw. The memory address is a 16-bit address formed by adding the contents of the program counter to the offset value

✓ **Answer**

False, it is offset from the base register, not the program counter.

3

Two processors that implement the same ISA must have identical performance for the same program

✓ **Answer**

False, different clock speeds, caching topology, or physical size of node can all influence performance when running on a physical processor.

4

In a system optimized for high throughput, the elapsed time of an individual program may increase even though overall system performance improves.

✓ **Answer**

True, high throughput optimization often means processing multiple things at once with a scheduler, which means that sometimes single threaded or single process tasks can take longer due to constant context switching.

5

A system with a lower average CPI is guaranteed to have better performance than a system with a higher CPI

✓ **Answer**

Just because your CPI is low doesn't necessarily mean that its clock speed is comparable too. If a higher CPI system has a significantly higher clock speed, then it may still be faster than the low CPI system.

6

Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (class A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2. Given a program with a dynamic instruction count of one million instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D, which is faster: P1 or P2?

a

What is the global CPI for each implementation?

✓ **Answer**

$$G_{CPI P_1} = 1(0.1) + 2(0.2) + 3(0.5) + 3(0.2) = 2.6$$

$$G_{CPI P_2} = 2(0.1) + 2(0.2) + 2(0.5) + 2(0.2) = 2.0$$

b

Find the clock cycles required in both cases.

✓ **Answer**

$$1000000(2.6) = 2600000$$

$$1000000(2.0) = 2000000$$

P_2 is faster.

7

```
addi  $t0, $s6, 4
add   $t1, $s6, $0
sw    $t1, 0($t0)
lw    $t0, 0($t0)
add   $s0, $t1, $t0
```

a

Translate the following MIPS code to C. Assume that the variable f is assigned to register \$s0 and the base address of the array A is in register \$s6.

```
A[1] = (int)A;  
f = (int)A + A[1];
```

b

For each MIPS instruction in part (a), show the value of the opcode (op), source register (rs) and funct field, and destination register (rd) fields. For the I-type instructions, show the value of the immediate field, and for the R-type instructions, show the value of the second source register (rt).

Instruction	Type	op	rs	rt	rd	funct	imm
addi \$t0, \$s6, 4	I	8	22	8			4
add \$t1, \$s6, \$0	R	0	22	0	9	32	
sw \$t1, 0(\$t0)	I	43	8	9			0
lw \$t0, 0(\$t0)	I	35	8	8			0
add \$s0, \$t1, \$t0	R	0	8	9	16	32	

8

Assume that we would like to expand the MIPS register file to 128 registers and expand the instruction set to contain four times as many instructions.

a

How would this affect the size of each of the bit fields in the R-type instructions?

✓ **Answer**

opcodes: add 2 bits (4x more)

register addresses: add 2 bits (32 -> 128 bits) (rs, rt, rd)

shamt, funct should remain the same unless you would like to add additional capabilities.

8 bits added total.

b

How would this affect the size of each of the bit fields in the I-type instructions?

✓ **Answer**

The same would be true for I-types, ops +2, reg +2 each

6 bits added total.

C

How could each of two proposed changes decrease the size of an MIPS assembly program? On the other hand, how could the proposed change increase the size of an MIPS assembly program?

✓ Answer

More capable instructions may be able to reduce the number of instructions total within a program, possibly decreasing the size of the total program.

Since each instruction is larger, it would likely increase the size as you may not be using much fewer instructions total.

9

Translate the following C code to MIPS assembly code using the instructions we learned in class. Assume that the values of a, b, i, and j are in registers \$s0, \$s1, \$t0, and \$t1, respectively. Also, assume that register \$s2 holds the base address of the array D

```
for (i=0; i<a; i++)
    for (j=0; j<b; j++)
        D[4*j] = i + j;
```



```
t0 = 0
outer_loop:
if (t0 ≥ s0) jmp end_outer
t1 = 0
inner_loop:
if (t1 ≥ s1) jmp end_inner
s2[4*t1] = t0+t1;
t1++;
jmp inner_loop
end_inner:
t0++;
jmp outer_loop
end_outer:
```

```
addi $t0, $0, 0
outer_loop:
bge $t0, $s0, end_outer
addi $t1, $0, 0
inner_loop:
bge $t1, $s1, end_inner
add $s2, $t0, $t1
```

```

sll $t3, $t1, 4
add $t3, $t3, $s2
sw $t2, 0($t3)
addi $t1, $t1, 1
j inner_loop
end_inner:
addi $t0, $t0, 1
j outer_loop
end_outer:

```

10

Implement the following C code in MIPS assembly.

```

int fib(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}

```



Return value *v0*

n in *a0*

```

fib:
addi $sp, $sp, -12
sw $ra, 8($sp)
sw $a0, 4($sp)

beq $a0, $0, base0
li $t0, 1
beq $a0, $t0, base1

addi $a0, $a0, -1
jal fib
sw $v0, 0($sp)

lw $a0, 4($sp)
addi $a0, $a0, -2
jal fib

lw $t1, 0($sp)

```

```
add $v0, $v0, $t1
```

```
j finish
```

```
base0:
```

```
li $v0, 0
```

```
j finish
```

```
base1:
```

```
li $v0, 1
```

```
finish:
```

```
lw $ra, 8($sp)
```

```
addi $sp, $sp, 12
```

```
jr $ra
```